Ira A. Fulton Schools of Engineering
Arizona State University

CSE 598: Intro to Deep Learning in Visual Computing

GANs for MNIST Dataset

Purpose

The purpose of this project is to explore and implement a Generative Adversarial Network (GAN), a popular generative neural network widely applied in computer vision tasks. You will be implementing GAN on MNIST data and generate images that resemble the digits from the MNIST dataset.

Objectives

Learners will be able to

- Acquire an understanding of the structure and training dynamics of GANs.
- Understand the analogy of a Discriminator and a Generator.

Technology Requirements

- GPU environment
- Jupyter Notebooks
- Python3 (Python 3.8 and above)
- PyTorch
- Torchvision
- Numpy
- Matplotlib

Directions

Accessing ZyLabs

You will complete and submit your work through zyBooks's zyLabs. Follow the directions to correctly access the provided workspace:

1. Go to the Canvas project, "**Submission: GANs for MNIST Dataset project**"
2. Click the "**Load Submission…in new window**" button.

3. Once in ZyLabs, click the **green button** in the Jupyter Notebook to get started.
4. Review the directions and resources provided in the description.
5. When ready, review the provided code and develop your work where instructed.

Project Directions

The GAN works by training a pair of networks, a Generator and a Discriminator, with competing loss terms. As an analogy, we can think of these networks as an art-forger and the other, an art-expert. In GAN literature the Generator is the art-forger and the Discriminator is the art-expert. The Generator is trained to produce fake images (forgeries) to deceive the art expert (Discriminator). The Discriminator which receives both the real images and fake images tries to distinguish between them to identify the fake images. The Generator uses the feedback from the Discriminator to improve its generation. Both models are trained simultaneously and are always in competition with each other. This competition between the Generator and Discriminator drives them to improve their respective models continuously. The model converges when the Generator produces fake images that are indistinguishable from the real images.

In this setup, the Generator does not have access to the real images whereas the Discriminator has access to both the real and the generated fake images.

Let us define Discriminator D which takes an image as input and produces a number **(0/1)** as output and Generator G which takes random noise as input and outputs a fake image. In practice, G and D are trained alternately i.e., For a fixed generator G, the Discriminator D is trained to classify the training data as real (output a value close to 1) or fake(output a value close to 0). Subsequently, we freeze the Discriminator and train Generator G to produce an image (fake) that outputs a value close to 1 (real) when passed through the Discriminator D. Thus, if the Generator is perfectly trained then the Discriminator D will be maximally confused by the images generated by G and predict 0.5 for all the inputs.

To implement a GAN, we require 5 components:

- Real Dataset (real distribution)
- Low dimensional random noise that is input to the Generator to produce fake images
- Generator that generates fake images
- Discriminator that acts as an expert to distinguish real and fake images.
- Training loop where the competition occurs and models better themselves.

Generator Architecture

Define a Generator with the following architecture.

- Linear layer (noise_dim -> 256)
- LeakyReLU (works well for the Generators, we will use negative_slope=2)
- Linear Layer (256 -> 512)
- LeakyReLU
- Linear Layer (512 -> 1024)
- LeakyReLU
- Linear Layer (1024 -> 784) (784 is the MNIST image size 28*28)
- TanH (To scale the generated images to [-1,1], the same as real images)
- LeakyRELU: https://pytorch.org/docs/stable/nn.html#leakyrelu
- Fully connected layer: https://pytorch.org/docs/stable/nn.html#linear
- TanH activation: https://pytorch.org/docs/stable/nn.html#tanh

Discriminator Architecture

Define a Discriminator with the following architecture.

- Linear Layer (input_size -> 512)
- LeakyReLU with negative slope = 0.2
- Linear Layer (512 -> 256)
- LeakyReLU with negative slope = 0.2
- Linear Layer (256 -> 1)

Binary Cross Entropy Loss

You will need to use the Binary cross entropy loss function to train the GAN. The loss function includes sigmoid activation followed by logistic loss. This allows us to distinguish between real and fake images.

Binary cross entropy loss with logits: https://pytorch.org/docs/stable/nn.html#bcewithlogitsloss

Discriminator Loss

Define the objective function for the Discriminator. It takes as input the logits (outputs of the Discriminator) and the labels (real or fake). It uses the BCEWithLogitsLoss() to compute the loss in classification.

Generator Loss

Define the objective function for the Generator. It takes as input the logits (outputs of the Discriminator) for the fake images it has generated and the labels (real). It uses the BCEWithLogitsLoss() to compute the loss in classification. The Generator expects the logits for the fake images it has generated to be close to 1 (real). If that is not the case, the Generatro corrects itself with the loss

GAN Training

Optimizers for training the Generator and the Discriminator.

Adam Optimizer: https://pytorch.org/docs/stable/optim.html#torch.optim.Adam

Feel free to adjust the optimizer settings.

Discriminator Optimization (D-Step)

- Clear Discriminator optimizer gradients.
- Estimate real image logits with the Discriminator.
- Generate fake images using the Generator and detach them to prevent Generator gradient computation.
- Estimate fake image logits with the Discriminator.
- Calculate Discriminator loss using the DLoss function.
- Backpropagate through the graph to compute gradients.
- Update Discriminator parameters.

Generator Optimization (G-Step)

- Clear Generator gradients.
- Generate fake images with the Generator.

- Estimate fake image logits with the Discriminator.
- Calculate Generator loss using the GLoss function.
- Backpropagate through the graph to compute gradients.
- Update Generator parameters.

Submission Directions for Project Deliverables

You must complete and submit your work through zyBooks's zyLabs to receive credit for the project:

1. To get started, use the provided Jupyter Notebook in your workspace.
2. All necessary datasets are already loaded into the workspace.
3. Execute your code by clicking the "**Run**" button in top menu bar.
4. When you are ready to submit your completed work, click on "**Submit for grading**" located on the bottom left from the notebook.
5. You will know you have completed the project when feedback appears below the notebook.
6. If needed: to resubmit the project in zyLabs
    a. Edit your work in the provided workspace.
    b. Run your code again.
    c. Click "**Submit for grading**" again at the bottom of the screen.

Your submission will be reviewed by the course team and then, after the due date has passed, your score will be populated from zyBooks into your course grade.

Evaluation

This project is auto-graded. Each test case has points assigned to it. Please review the notebook to see the points assigned for each test case. A percentage score will be passed to Canvas based on your score.

This assignment has both auto-graded and manually-graded test cases. There are a total of five (5) test cases.

- Four (4) of the five (5) test cases are auto-graded.
- The last test case will be manually graded.

Please review the notebook to see the points assigned for each test case. A percentage score will be passed to Canvas based on your score.